

Zona Docker

Volcado a PDF

Table of contents

1. Zona Docker	4
1.1 1. Intro	4
1.2 2. Amplia	4
2. Tutorial Docker	5
2.1 1. Instalar docker engine	5
2.2 2. Realizar paso a paso el tutorial docker 101	5
2.3 3. Típicos problemas en el aula	5
3. Servidores	6
3.1 Servidores Web	6
3.2 Bases de Datos	8
3.3 Servidor de Aplicaciones	10
3.4 Ampliación	12
4. Ejercicios	15
4.1 Servidores	15
4.2 Crear su propia imagen	15
4.3 Configuraciones multicontenedor	15
4.4 Traefik	15
5. Traefik	16
5.1 1. Introducción	16
5.2 2. Iniciar Traefik	17
5.3 3. Descubriendo servicios	18
5.4 Ampliando el ejemplo inicial	18
5.5 4. Escalar el servicio: balanceo de carga	21
5.6 5. Ejercicio propuesto	21
5.7 6. Enlaces	21
5.8 7. Anexo: Fichero <code>docker-compose.yml</code> completo	21
6. Cluster: Docker Swarm	23
6.1 1. Introducción	23
6.2 2. Preparación del <i>swarm</i>	23
6.3 3. Desplegar servicio	24
6.4 4. Herramienta gráfica: portainer	26
6.5 5. Anexos	27
7. Fuentes ejemplos o ejercicios	29
7.1 Código ejemplos de sitios web	29
7.2 Código ejemplo tomcat	29

7.3	Scripts SQL	29
7.4	Docker Swarm	29
8.	Otros	30
8.1	Encadenar arranque contenedores	30
8.2	Complementos y ayudas	32

1. Zona Docker

1.1 1. Intro

Los contenidos de esta zona o bloque son:

- Se inicia con tutorial docker de la propia empresa [docker.com](https://docs.docker.com/)
- Se refuerza con ejercicios de despliegue de servidores, CMS, etc.
- Se introduce el uso de `traefik` como proxy inverso en un sistema en local de contenedores,
- Se muestra un cluster `docker swarm`.

1.2 2. Amplia

- Desplegar un entorno de contenedores docker con `traefik` en una instancia AWS con certificados SSL. Para ello:
 - a. [Uso de AWS](#)
 - b. [Traefik y https](#)

2. Tutorial Docker

2.1 1. Instalar docker engine

1. Instalar docker engine en la máquina virtual: [Install Docker Engine on Debian](#)
2. Realizar la post-instalación: [Manage Docker as a non-root user](#)

2.2 2. Realizar paso a paso el tutorial docker 101

If you wish to run the tutorial, you can use the following command after installing

```
docker run -d -p 80:80 docker/getting-started
```

Once it has started, you can open your browser to <http://localhost>.

Además dispone en la plataforma de un curso de *Introducción a Docker*.

2.3 3. Típicos problemas en el aula

- Levantar un contenedor con un nombre que ya está siendo usado por otro contenedor (activo o no).
- Levantar un contenedor mapeando un puerto del host que ya está siendo usado por otro contenedor.
- Ojo con la "cache" del navegador, puede ser necesario forzar la recarga de la página, o usar el comando `curl`, etc.
- Existen dos versiones del tutorial docker, y el código de la aplicación de ejemplo que se utiliza es diferente en cada versión. No puede mezclar la aplicación de un tutorial con los comandos o el `docker-compose.yml` del otro. Las dos versiones del tutorial son:
 - `docker/getting-started:latest` (usa node 18 y mysql 8)
 - `docker/getting-started:pwd` (usa node 10 y mysql 5.7)
- Cuidado al crear y usar un volumen para una versión de una base de datos e intentar usar ese volumen con otra versión de la base de datos (o con otra base de datos).

3. Servidores

3.1 Servidores Web

Una vez que conoce el funcionamiento básico de la tecnología de contenedores puede desplegar servicios (servidores web, bases de datos, gestores de contenidos, etc.) de una forma rápida y sencilla.

3.1.1 1. Uso básico

Puede probar a desplegar un servidor web nginx o apache (httpd).

```
alu@xd13:~/DW/apache$ docker run -d -p 80:80 --name web httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
...
Digest: sha256:ca375ab8ef2cb8bede6b1bb97a943cce7f0a304d5459c085235b47bc2dccb98cd
Status: Downloaded newer image for httpd:latest
04464ccf11e5fa9b3e809eb0f23fc32f6737d825a29c49cf60497a8570bd2758
alu@xd13:~/DW/apache$
```

Y para probar basta usar `curl` y comprobar que se muestra la página por defecto del servidor Apache:

```
alu@xd13:~/DW/apache$ curl localhost
<html><body><h1>It works!</h1></body></html>
alu@xd13:~/DW/apache$
```

Puede modificar el fichero `/etc/hosts` para "simular" resolución de nombres y crear un dominio `sitio1.internal` (sobre el uso de TLDs privados).

Edite y añada una línea en dicho fichero similar a `127.0.0.1 sitio1.internal`

```
alu@xd13:~/DW/apache$ sudo nano /etc/hosts
[sudo] contraseña para alu:
alu@xd13:~/DW/apache$ cat /etc/hosts
127.0.0.1 sitio1.internal

127.0.0.1 localhost
127.0.1.1 xd13
...
alu@xd13:~/DW/apache$
```

Y ahora puede usar el nombre del dominio "creado":

```
alu@xd13:~/DW/apache$ curl sitio1.internal
<html><body><h1>It works!</h1></body></html>
alu@xd13:~/DW/apache$ firefox sitio1.internal &
```

3.1.2 2. Con sitio web personalizado

Para desplegar un sitio web desarrollado por usted debe consultar en la documentación de DockerHub donde debe copiar (o montar) su sitio web. En el caso de la imagen `httpd` es el directorio `/usr/local/apache2/htdocs/`.

Para ello cree un directorio de nombre `html`, y dentro coloque un sitio web de prueba con un fichero `index.html` y un fichero de estilos CSS:

```
alu@xd13:~/DW/apache$ mkdir html
alu@xd13:~/DW/apache$ code html
# Se crea fichero html y hoja de estilos
alu@xd13:~/DW/apache$ ls -R html
html:
css index.html

html/css:
estilos.css
alu@xd13:~/DW/apache$
```

Para ejecutar y probar basta hacer un bind mount de `html` a `/usr/local/apache2/htdocs/` añadiendo la opción `-v ./html:/usr/local/apache2/htdocs/`:

```
alu@xd13:~/DW/apache$ docker run -d -p 80:80 --name web -v ./html:/usr/local/apache2/htdocs/ httpd
4e78397f71b1151f7bfff33281fbcfac976c6d97ee12cc5999008e596aac7ff98
alu@xd13:~/DW/apache$ curl sitio1.internal
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Apache y Docker</title>
  <link rel="stylesheet" href="css/estilos.css">
</head>
<body>
  <h1>Sitio web Apache personalizado</h1>
</body>
</html>alu@xd13:~/DW/apache$
```

Y para mejorar esta solución puede crear un fichero compose:

compose.yml

```
services:
  apache:
    image: httpd:2.4
    container_name: miweb
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/local/apache2/htdocs
    restart: always
```

3.1.3 3. Imagen con aplicación

En ese caso necesita un fichero Dockerfile que a partir de la imagen de apache copia su web en `/usr/local/apache2/htdocs`:

Dockerfile

```
FROM httpd:2.4
COPY ./html /usr/local/apache2/htdocs
```

```
alu@xd13:~/DW/apache$ ls
docker-compose.yml Dockerfile html
alu@xd13:~/DW/apache$ docker build -t miapp .
[+] Building 2.4s (7/7) FINISHED          docker:default
=> [internal] load build definition from Dockerfile          0.1s
=> => transferring dockerfile: 90B                          0.0s
=> [internal] load metadata for docker.io/library/httpd:2.4  1.8s
=> [internal] load .dockerignore                             0.1s
=> => transferring context: 2B                                0.0s
=> CACHED [1/2] FROM docker.io/library/httpd:2.4@sha256:ca375ab8ef 0.0s
=> [internal] load build context                             0.1s
=> => transferring context: 504B                              0.0s
=> [2/2] COPY ./html /usr/local/apache2/htdocs              0.1s
=> exporting to image                                       0.2s
=> => exporting layers                                       0.1s
=> => writing image sha256:e77f76dd6733523aa6b18b1f37bd9c25ba48aad 0.0s
=> => naming to docker.io/library/miapp                     0.0s
alu@xd13:~/DW/apache$
```

Y ya puede lanzar su propia imagen con su sitio web incorporado:

```
alu@xd13:~/DW/apache$ docker run -d -p 80:80 --name miweb miapp:latest
2cc9f6cbb96fd7a89e847dc635f88b9b48ba7f6895aff3b0dbc6edb401b9300
alu@xd13:~/DW/apache$ curl sitio1.internal
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Apache y Docker</title>
  <link rel="stylesheet" href="css/estilos.css">
</head>
<body>

  <h1>Sitio web Apache personalizado</h1>

</body>
</html>alu@xd13:~/DW/apache$
```

3.2 Bases de Datos

Una vez que conoce el funcionamiento básico de la tecnología de contenedores puede desplegar servicios (servidores web, bases de datos, gestores de contenidos, etc.) de una forma rápida y sencilla.

3.2.1 1. Desde CLI

En este apartado va a desplegar un contenedor usando la imagen de `mysql`. Si consulta la documentación de dicha imagen verá que la base de datos se puede configurar a través del uso de variables de entorno. En este ejemplo inicial se utilizará la variable de entorno `MYSQL_ROOT_PASSWORD` para indicarle al contenedor la clave del usuario root.

Si usa la línea de comandos las variables de entorno se pasan con la opción `-e`, en este caso `-e MYSQL_ROOT_PASSWORD=miclave`.

```
docker run -d --name mibd -e MYSQL_ROOT_PASSWORD=miclave mysql
```

Para comprobar que la BD está operativa se inicia una shell en el contenedor con el comando `docker exec` y dentro se ejecuta el cliente `mysql`:

```
alu@xd13:~/DW$ docker exec -it mibd bash
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 9.4.0 MySQL Community Server - GPL
...

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database           |
+-----+
| information_schema |
| mysql               |
| performance_schema |
| sys                 |
+-----+
4 rows in set (0.015 sec)

mysql>
```

O todo en uno ejecutando el cliente `mysql` desde `docker exec`:

```
alu@xd13:~/DW$ docker exec -it mibd mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 9.4.0 MySQL Community Server - GPL
...

mysql>
```

3.2.2 2. Con compose

De una forma similar a los servidores web puede crear un fichero `compose` y en este caso pasar la información de las variables de entorno:

```
compose.yaml

# Use root/example as user/password credentials
services:
  db:
    image: mysql
    container_name: mibd
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: example
    # (this is just an example, not intended to be a production configuration)
```

3.2.3 3. Con gestor web

Puede añadir al `compose` anterior el servicio `adminer` (o `phpmyadmin`):

```
...
adminer:
  image: adminer
  restart: always
  ports:
    - 8080:8080
```

3.2.4 4. Persistencia

Y para terminar tenga en cuenta la persistencia de los datos, necesita volúmenes para mantener la información [Where to Store Data](#)

3.3 Servidor de Aplicaciones

3.3.1 1. ¿Qué es un servidor de aplicaciones?

Un servidor de aplicaciones es un software que proporciona un entorno para ejecutar aplicaciones, generalmente en el lado del servidor. Sus funciones incluyen:

- Gestionar peticiones HTTP desde clientes (navegadores).
- Ejecutar código Java, servlets, JSP o frameworks web.
- Ofrecer servicios comunes: seguridad, gestión de sesiones, conexión a bases de datos, logging, etc.

👉 Ejemplos de servidores de aplicaciones:

- Apache Tomcat (muy usado para Java EE o Jakarta EE Web Profile)
- WildFly (antes JBoss)
- GlassFish / Payara
- Jetty

3.3.2 2. Apache Tomcat

Tomcat es un servidor open source desarrollado por la Apache Software Foundation. Implementa las especificaciones Servlet y JSP.

Características principales:

- Ligero y fácil de configurar.
- Ideal para desplegar aplicaciones Java web (archivos `.war`).
- Se puede ejecutar como servicio o en contenedor Docker.
- Accesible por defecto en el puerto 8080.

3.3.3 3. Tomcat con Docker

En lugar de instalar Tomcat manualmente, podemos usar una imagen Docker (`tomcat:10` o `tomcat:9`). Ventajas:

- No hay que configurar Java ni Tomcat en el sistema.
- Entorno limpio, reproducible y aislado.
- Ideal para entornos educativos y despliegue rápido.

Requisitos previos

- Tener Docker instalado y funcionando,
- Disponer de una aplicación web Java empaquetada en un archivo `.war`.

3.3.4 4. Práctica

4.1. La aplicación

Crear un directorio:

```
mkdir tomcat-docker
cd tomcat-docker
```

Se le proporciona en este ejemplo una aplicación desarrollada en JSP que solicita que el usuario adivine un nombre. Este es el fichero que debe descargar:

Fichero [guess-number.zip](#)

Ahora basta descomprimir y crear el fichero war:

```
unzip guess-number.zip
cd guess-number
jar cvf guess-number.war *.jsp
```

4.2. Crear la imagen

Usar un Dockerfile similar a este que debe almacenar en la carpeta `tomcat-docker`

Dockerfile

```
# Usar la imagen oficial de Tomcat
FROM tomcat
# FROM tomcat:10-jdk17

# Copiar nuestra app al directorio webapps
COPY ./guess-number/guess-number.war /usr/local/tomcat/webapps/ROOT.war

# Exponer el puerto 8080
EXPOSE 8080
```

Y construir la imagen

```
docker build -t mitomcat-app .
```

4.3. Probarla



Ejecuta en tu terminal

```
docker run -d -p 8080:8080 --name tomcat-app mitomcat-app
firefox http://localhost:8080
```

Deberías ver la aplicación Java funcionando dentro del contenedor Tomcat.

3.3.5 5. Con compose

En este caso no se crea una imagen sino que se usa la de tomcat, y un volumen para la aplicación. Usar un compose similar a este:

compose.yaml

```
services:
  tomcat:
    image: tomcat
    container_name: mitomcat
    ports:
      - "8080:8080"
      - ./guess-number/guess-number.war:/usr/local/tomcat/webapps/ROOT.war
    restart: unless-stopped
```

Y para ejecutar

```
docker compose up -d
```

3.4 Ampliación

Puede ver el curso de OW sobre Apache: [Servidor Web Apache 2.4](#). En el se explican los conceptos de ficheros de configuración, y opciones como virtual hosting, mapeo de URLs, SSL, módulos, etc.

Se muestran a continuación ejemplos de algunas de ellas.

3.4.1 1. Virtual hosts

1.1. Añadir dominios locales al sistema

Edita el archivo `/etc/hosts`

Agregue las siguientes líneas :

```
127.0.0.1    sitio1.local
127.0.0.1    sitio2.local
```

1.2. Estructura directorios

```
mkdir apache-docker
cd apache-docker
```

Y cree esta estructura en su interior. Otra opción es descargar el fichero comprimido:

```
apache-docker/
├── docker-compose.yml
├── html/
│   ├── sitio1/
│   │   └── index.html
│   └── sitio2/
│       └── index.html
└── conf/
    ├── httpd-vhosts.conf
    └── httpd.conf
```

1.3. Configuración apache

Necesita los ficheros:

- `httpd.conf` , configuración genral de apache
- `httpd-vhosts.conf` , configuración de los hosts virtuales `httpd-vhosts.conf`

1.4. Compose

Necesitará un compose similar a este:

```
services:
  apache:
    image: httpd:2.4
    container_name: apache_local
    ports:
      - "80:80"
    volumes:
      # Directorio con el los dos sitios web
      - ../html:/usr/local/apache2/htdocs
      # Ficheros de configuración de apache
      - ../conf/vhosts.conf:/usr/local/apache2/conf/extra/httpd-vhosts.conf
      - ../conf/httpd.conf:/usr/local/apache2/conf/httpd.conf
```

1.5. Probar

```
docker compose up -d
curl sitio1.local
curl sitio2.local
```

3.4.2.2. SSL (autofirmado)

2.1. Entorno

Se añaden a continuación una carpeta `certs` y un nuevo archivo de configuración `vhosts-ssl.conf` y la estructura quedará:

```

apache-docker/
├── docker-compose.yml
├── conf/
│   ├── httpd.conf
│   ├── vhosts.conf
│   └── vhosts-ssl.conf
├── certs/
├── html/
│   ├── sitio1/
│   └── sitio2/

```

2.2. Generar certificados autofirmados

Ejecutar en la carpeta raíz del proyecto:

```

# Crear directorio
apache-docker$ mkdir certs

# Crear Certificado de sitio1
apache-docker$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/sitio1.local.key \
-out certs/sitio1.local.crt \
-subj "/C=ES/ST=Madrid/L=Madrid/O=LocalDev/CN=sitio1.local"

# Crear Certificado de sitio2
apache-docker$ openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/sitio2.local.key \
-out certs/sitio2.local.crt \
-subj "/C=ES/ST=Madrid/L=Madrid/O=LocalDev/CN=sitio2.local"

```

2.3. Fichero configuración

Se añade el fichero `vhosts-ssl.conf` del que se muestra un extracto:

```

Listen 443
LoadModule ssl_module modules/mod_ssl.so
LoadModule socache_shmcb_module modules/mod_socache_shmcb.so

<VirtualHost *:443>
    ServerAdmin webmaster@sitio1.local
    ServerName sitio1.local
    DocumentRoot "/usr/local/apache2/htdocs/sitio1"

    SSLEngine on
    SSLCertificateFile "/usr/local/apache2/conf/certs/sitio1.local.crt"
    SSLCertificateKeyFile "/usr/local/apache2/conf/certs/sitio1.local.key"

    <Directory "/usr/local/apache2/htdocs/sitio1">
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog "logs/sitio1-ssl-error.log"
    CustomLog "logs/sitio1-ssl-access.log" common
</VirtualHost>
...

```

Además hay que modificar `httpd.conf` y añadir al final del mismo estas líneas:

```

...
# Incluir también la configuración SSL
Include conf/extra/httpd-vhosts-ssl.conf

```

2.4. Modificar compose

Se modifica el fichero compose para:

- Añadir redirección del puerto 443
- Añadir volúmenes:
- con una ruta a los certificados creados
- con el nuevo fichero de configuración `vhosts-ssl.conf`
-

```
services:
  apache:
    image: httpd:2.4
    container_name: apache_local
    ports:
      - "80:80"
      # SSL
      - "443:443"
    volumes:
      - ./html:/usr/local/apache2/htdocs
      - ./conf/vhosts.conf:/usr/local/apache2/conf/extra/httpd-vhosts.conf
      - ./conf/httpd.conf:/usr/local/apache2/conf/httpd.conf
      # SSL
      - ./conf/vhosts-ssl.conf:/usr/local/apache2/conf/extra/httpd-vhosts-ssl.conf
      - ./certs:/usr/local/apache2/conf/certs
```

2.5. Probar

Tenga en cuenta que al ser certificados autofirmados el navegador no mostrará la página y avisará de un posible fallo de seguridad. Otra posibilidad es usar el comando `curl` con la opción `-k`:

```
$ curl https://sitio2.local
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html
...
$ curl -k https://sitio2.local
<!DOCTYPE html>
...
    <h1>Sitio 2</h1>
</body>
</html>
$
```

Opciones de utilidad con curl:

Opción	Descripción
-L	Sigue las redirecciones.
-k	Ignora errores de certificado.
-I	Muestra solo los encabezados.
-v	Modo detallado (verbose).

2.6. Ampliación

Se le propone modificar la configuración del servidor para que las peticiones al puerto 80 se redirijan al puerto 443 (redirección HTTP a HTTPS)

4. Ejercicios

4.1 Servidores

1. Lanzar contenedor con servidor web nginx. Probar. Parar y borrar.
2. Lanzar contenedor con servidor web nginx mapeando un directorio del host como sitio web (usar por ejemplo fuentes del juego 2048). Probar.
3. Lanzar segundo contenedor nginx usando un puerto diferente. Probar. Parar y borrar.
4. Lanzar contenedor con servidor web apache usando fuente del fichero `Netflix.zip`. Probar. Parar y borrar.
5. Lanzar contenedor con **servidor web con php**. Ver variantes de la imagen php con apache. Mapear un directorio local que contenga un fichero `index.php` de prueba. Probar. Parar y borrar.
6. Lanzar servidor con BD `mysql`. Acceder luego a la consola `mysql` del contenedor usando `docker exec`. Parar y borrar.
7. Lanzar servidor con BD `mariadb` usando un volumen docker. Hacerlo generando una clave de root aleatoria.
8. a. Investigar: Se propone que al lanzar un contenedor con un motor de BD (pe. `mysql`) se cree una base de datos y se rellene con datos iniciales.
Pista: `docker-entrypoint-initdb.d` (puede usar los ficheros de ejemplo).

4.2 Crear su propia imagen

1. Crear una imagen propia con el juego 2028. Antes modificar el `title` de la página principal para indicar vuestro nombre. Probar. Subir a DockerHub.

4.3 Configuraciones multicontenedor

1. Crear un fichero `docker-compose` para lanzar una base de datos y el gestor `adminer` (o `phpmyadmin`) en dos contenedores. Probar el acceso desde `adminer` a la BD (ejemplo en la info de la imagen `adminer`).
2. Modificar anterior para almacenar la BD en un volumen. Generar info desde `adminer` y comprobar que persiste tras reiniciar el contenedor
3. Crear/buscar fichero `docker-compose` para lanzar Wordpress en dos contenedores (aplicación y BD). Probar.
4. Revisar apartado anterior: Añadir una entrada y un tema nuevo a Wordpress. Probar. Detener y borrar los contenedores. Reiniciar el servicio y comprobar que permanece la entrada y el tema. Si no, solucionar.
5. Opcional: probar configuración multicontenedor de algún otro CMS como `joomla` o `drupal`.

4.4 Traefik

1. Añadir a ejemplos de clase el servicio `jugar` que lanza la imagen del juego 2048 construida previamente. Debe responder en la dirección `http://jugar.sitio1.internal`.
2. Añadir a ejemplo de clase el CMS Wordpress. Debe responder en la dirección `http://wordpress.midominio.internal`.

5. Traefik

5.1 1. Introducción

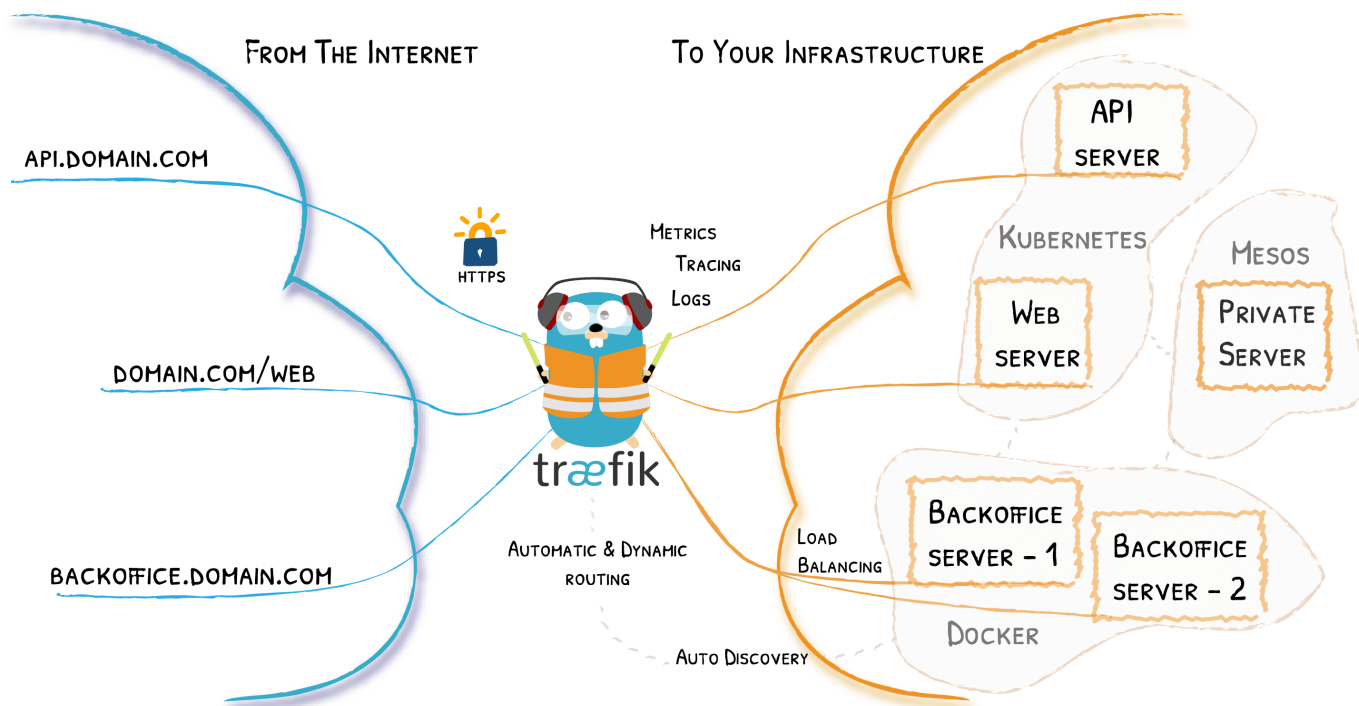
Un proxy inverso es un servidor que se sitúa delante de uno o varios servidores web, e intercepta las solicitudes de los clientes. Esto es diferente de un proxy de reenvío, en el que el proxy se sitúa delante de los clientes. Con un proxy inverso, cuando los clientes envían solicitudes al servidor de origen de un sitio web, el servidor de proxy inverso intercepta esas solicitudes en el perímetro de la red. El servidor proxy inverso enviará entonces las solicitudes al servidor de origen y recibirá las respuestas del servidor de origen.

La diferencia entre un proxy de reenvío y uno inverso es sutil pero importante. Una forma simplificada de resumirla sería decir que un proxy de reenvío se sitúa delante de un cliente y se asegura de que ningún servidor de origen se comunique nunca directamente con ese cliente específico. Por otro lado, un proxy inverso se sitúa delante de un servidor de origen y se asegura de que ningún cliente se comunique nunca directamente con ese servidor de origen.

5.1.1 1.1. Traefik como proxy inverso

Nginx o Apache pueden cumplir perfectamente esta función, aunque hoy en día hay una nueva categoría de proxys reversos pensados para microservicios y también diseñados para “dialogar” con plataformas de contenedores como Docker, Kubernetes, etc. Este es el caso de Traefik.

Traefik is an open-source Edge Router that makes publishing your services a fun and easy experience. It receives requests on behalf of your system and finds out which components are responsible for handling them.



What sets Traefik apart, besides its many features, is that it automatically discovers the right configuration for your services. The magic happens when Traefik inspects your infrastructure, where it finds relevant information and discovers which service serves which request.

The main features include dynamic configuration, automatic service discovery, and support for multiple backends and protocols.

Traefik is based on the concept of EntryPoints, Routers, Middlewares and Services.

- **EntryPoints:** EntryPoints are the network entry points into Traefik. They define the port which will receive the packets, and whether to listen for TCP or UDP.
- **Routers:** A router is in charge of connecting incoming requests to the services that can handle them.
- **Middlewares:** Attached to the routers, middlewares can modify the requests or responses before they are sent to your service
- **Services:** Services are responsible for configuring how to reach the actual services that will eventually handle the incoming requests.

1.1.1. Edge Router¶

Traefik is an Edge Router, it means that it's the door to your platform, and that it intercepts and routes every incoming request: it knows all the logic and every rule that determine which services handle which requests (based on the path, the host, headers, etc.).

1.1.2. Auto Service Discovery¶

Where traditionally edge routers (or reverse proxies) need a configuration file that contains every possible route to your services, Traefik gets them from the services themselves.

Deploying your services, you attach information that tells Traefik the characteristics of the requests the services can handle.

5.2.2. Iniciar Traefik

Se parte del ejemplo que proporciona Traefik en su [guía rápida](#).

El fichero inicial sólo contiene el servicio de nombre `traefik` que lanza Traefik

```
services:
  traefik:
    image: traefik:v3.5
    command:
      - "--api.insecure=true"
```

```

- "--providers.docker=true"
- "--entrypoints.web.address=:80"
ports:
- "80:80"
- "8080:8080"
volumes:
- /var/run/docker.sock:/var/run/docker.sock

```

Inicie el servicio con el comando `docker compose` y puede acceder al *dashboard*:

Because we explicitly enabled insecure mode, the dashboard is reachable on port 8080 without authentication. **Do not enable this flag in production.** You can access the dashboard at: `http://localhost:8080/dashboard/`

5.3 3. Descubriendo servicios

5.3.1 3.1. Ejemplo inicial

La guía rápida nos propone añadir un servicio web de nombre *whoami* que muestra información del equipo sobre el que se ejecuta ese servicio en un fichero de nombre `whoami.yml`.

En Docker, las labels son una forma de añadir metadatos a los contenedores. Traefik usa estas etiquetas para descubrir automáticamente servicios y configurar dinámicamente las rutas, puertos, reglas, etc.

whoami.yml

```

# whoami.yml
services:
  whoami:
    image: traefik/whoami
    labels:
      - "traefik.http.routers.whoami.rule=Host(`whoami.localhost`)"

```

Apply the configuration:

```
docker-compose -f whoami.yml up -d
```

Test Your Setup

```
curl http://whoami.localhost
```

Detener los servicios:

```
docker compose up --remove-orphans -d
```

5.4 Ampliando el ejemplo inicial

Vamos a generar un ejemplo con una dirección "real" y añadir toda la configuración en un único fichero.

Este es el bloque a añadir en `docker-compose.yml`. Observe que se ha modificado el servicio `whoami` para usar el dominio `md.internal`:

```

whoami:
  # A container that exposes an API to show its IP address
  image: traefik/whoami
  labels:
    - "traefik.enable=true" # es necesario si la opción de traefik --providers.docker.exposedbydefault=false
    - "traefik.http.routers.whoami.entrypoints=http" # valor por defecto, se podrá obviar
    - "traefik.http.routers.whoami.rule=Host(`whoami.md.internal`)"

```

Y se ha añadido al ejemplo de la guía rápida las labels:

- `"traefik.enable=true"` : es una opción necesaria en el servicio si en la opciones de traefik estuviera activa la opción `--providers.docker.exposedbydefault=false`. Por defecto cuando Traefik se conecta al socket Docker expone automáticamente todos los contenedores como servicios. Es decir, si `--providers.docker.exposedbydefault` está en `true` (valor por defecto), cualquier contenedor puede ser accesible públicamente si tiene un puerto abierto. Con `traefik.enable` se controla dicha exposición a nivel de servicio.
- `"traefik.http.routers.whoami.entrypoints=web"`. En este caso esta opción no es necesaria ya que `web` y `websecure` son entrypoints ya definidos en Traefik cuando no defines explícitamente otros en la configuración (web se corresponde con el puerto 80).

If not specified, HTTP routers are attached to all entry points.

the entryPoints web (port 80), websecure (port 443), traefik (port 8080) and metrics (port 9100) are created by default. The entryPoints web, websecure are exposed by default using a Service.

Para iniciar los servicios

```
alu@xd13:~/pr/traefik$ docker compose up -d
```

Traefik descubre este nuevo servicio y lo añade a su configuración creando una ruta que enlaza la dirección `whoami.md.internal` al servicio.

```
alu@xd13:~/pr/traefik$ curl -H "Host:whoami.md.internal" localhost
Hostname: fc5ab2a1c07a
IP: 127.0.0.1
...
X-Real-IP: 172.18.0.1
alu@xd13:~/pr/traefik$
```

Observe que se prueba el servicio: `*` con el comando `curl * el parámetro -H "Host:whoami.md.internal"` (ya que la dirección `whoami.md.internal` no tiene asociada ninguna IP añadimos el campo `Host` en la petición HTTP) `*` como destino `localhost`: si intentamos acceder a la dirección `whoami.md.com` (en el navegador o haciendo ping) dará error ya que no existe ese host ni ese dominio y no hay ninguna IP asociada.

Para resolver la IP de un nombre de host se consulta primero el fichero `/etc/hosts` y si no hay correspondencia a los servidores DNS. Para hacer una prueba más cercana a la realidad se asocia en ese fichero el nombre `whoami.md.internal` a la dirección local `127.0.0.1`, Y ya podría usar ese nombre en sus pruebas en local con el navegador, etc.

```
alu@xd13:~/pr/traefik$ sudo nano /etc/hosts
alu@xd13:~/pr/traefik$ cat /etc/hosts
127.0.0.1      whoami.md.internal localhost
127.0.1.1      xd13
...
alu@xd13:~/pr/traefik$
```

Puede probar ahora con `ping` la resolución del nombre a IP:

```
alu@xd13:~/pr/traefik$ ping whoami.md.internal
PING whoami.md.internal (127.0.0.1) 56(84) bytes of data.
64 bytes from whoami.md.internal (127.0.0.1): icmp_seq=1 ttl=64 time=0.064 ms
64 bytes from whoami.md.internal (127.0.0.1): icmp_seq=2 ttl=64 time=0.088 ms
^C
--- whoami.md.internal ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1021ms
rtt min/avg/max/mdev = 0.064/0.076/0.088/0.012 ms
alu@xd13:~/pr/traefik$
```

Y ahora puede acceder con `curl` sin la opción `-H` (o desde el navegador web)

```
alu@xd13:~/pr/traefik$ curl whoami.md.internal
Hostname: 5d2d95b8f9b3
IP: 127.0.0.1
IP: 172.19.0.3
...
alu@xd13:~/pr/traefik$ firefox whoami.md.internal &
[1] 9050
alu@xd13:~/pr/traefik$
```

Añada ahora otras dos direcciones (que se usan en apartados posteriores) al fichero `/etc/hosts`:

- `web.md.internal` (se asociará a un servidor nginx)
- `php.otrodominio.internal` (se asociará a un servidor PHP)

```
alu@xd13:~/pr/traefik$ sudo nano /etc/hosts
alu@xd13:~/pr/traefik$ cat /etc/hosts
127.0.0.1      php.otrodominio.internal web.md.internal whoami.md.internal localhost
127.0.1.1      vm
...
alu@xd13:~/pr/traefik$
```

5.4.1 3.2. Servicio web nginx

En este apartado se añade un servidor web `nginx`. En este caso se asocia el host `web.md.internal` al servicio. El bloque a añadir en `docker-compose.yml` sería:

```
nginx:
  image: nginx
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.nginx.entrypoints=web"
    - "traefik.http.routers.nginx.rule=Host(`web.md.internal`)"
```

Observe que hay que darle un nombre a la "ruta", y se ha usado en las labels `entrypoints` y `rule` el mismo nombre del servicio. Para activar dicho servicio:

```
alu@xd13:~/pr/traefik$ docker compose up -d nginx
[+] Running 1/1
✓ Container p04_traefik-nginx-1 Started 0.0s
alu@xd13:~/pr/traefik$
```

Nuevamente Traefik descubre el servicio en tiempo real y lo añade a sus rutas. Ahora puede acceder a la dirección `web.md.internal` (con curl o firefox)

```
alu@xd13:~/pr/traefik$ curl web.md.internal
<!DOCTYPE html>
...
<body>
  <p>Página inicio servidor nginx. </p>
</body>
</html>
alu@xd13:~/pr/traefik$
```

5.4.2 3.3. Servidor php

En este apartado se añade un servidor web con PHP. Además se usa un TLD distinto (`otrodominio.internal`). Y como variación el servidor web-PHP mapea un directorio local al contenedor. En el directorio está el fichero `index.php` que muestra la IP y nombre del host. El contenido del fichero `index.php` es:

```
<?php echo 'Server IP: ' . $_SERVER['SERVER_ADDR'] . nl2br(" \n");?>
<?php echo 'Hostname: ' . gethostname() . nl2br(" \n");?>
```

Se añade el nuevo servicio a `docker-compose.yml`. Observe el nombre de las rutas usado (`phpserver`) y el valor de `Host`:

```
...
phpserver:
  image: php:apache
  volumes:
    - ./html:/var/www/html
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.phpserver.entrypoints=web"
    - "traefik.http.routers.phpserver.rule=Host(`php.otrodominio.internal`)"
```

Se lanza el servicio:

```
alu@xd13:~/pr/traefik$ docker compose up -d phpserver
[+] Running 1/1
✓ Container p04_traefik-phpserver-1 Started 0.1s
alu@xd13:~/pr/traefik$
```

Y pruebe el nuevo servicio (en consola y navegador):

```
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.5 <br />
Hostname: 93206dc48cea <br />
alu@xd13:~/pr/traefik$ firefox php.otrodominio.internal &
```

5.5 4. Escalar el servicio: balanceo de carga

Puede "escalar" este último servicio creando replicas con la opción `--scale`:

```
alu@xd13:~/pr/traefik$ docker compose up -d --scale phpserver=3
[+] Running 6/6
✓ Container p04_traefik-reverse-proxy-1 Running 0.0s
✓ Container p04_traefik-nginx-1 Running 0.0s
✓ Container p04_traefik-whoami-1 Running 0.0s
✓ Container p04_traefik-phpserver-1 Running 0.0s
✓ Container p04_traefik-phpserver-3 Started 0.0s
✓ Container p04_traefik-phpserver-2 Started 0.1s
alu@xd13:~/pr/traefik$
```

Si accede ahora al servicio puede comprobar cómo responden las distintas replicas ya que traefik hace balanceo de carga:

```
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.5 <br />
Hostname: 93206dc48cea <br />
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.6 <br />
Hostname: a23b9ac33c77 <br />
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.7 <br />
Hostname: 8ecfc43015ab <br />
alu@xd13:~/pr/traefik$ curl php.otrodominio.internal
Server IP: 172.20.0.5 <br />
Hostname: 93206dc48cea <br />
alu@xd13:~/pr/traefik$
```

5.6 5. Ejercicio propuesto

Añadir a traefik un servicio `Wordpres` en el dominio `miblog.example.internal`. Tenga en cuenta que son dos servicios: `wordpress` y la base de datos.

5.7 6. Enlaces

- [Quickstart de Traefik](#)
- [Traefik Docker Example and Introduction](#)

5.8 7. Anexo: Fichero `docker-compose.yml` completo

```
# compose.yml
services:

  traefik:
    image: traefik:v3.5
    command:
      - "--api.insecure=true"
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
    ports:
      - "80:80"
      - "8080:8080"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock

  whoami:
    # A container that exposes an API to show its IP address
    image: traefik/whoami
    labels:
      - "traefik.enable=true" # es necesario si la opción de traefik --providers.docker.exposedbydefault=false
      - "traefik.http.routers.whoami.entrypoints=web" # valor por defecto, se podrá obviar
      - "traefik.http.routers.whoami.rule=Host(`whoami.md.internal`)"

  nginx:
```

```
image: nginx
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.nginx.entrypoints=web"
  - "traefik.http.routers.nginx.rule=Host(`web.md.internal`)"

phpserver:
image: php:apache
volumes:
  - ./html:/var/www/html
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.phpserver.entrypoints=web"
  - "traefik.http.routers.phpserver.rule=Host(`php.otrodominio.internal`)"
```

Aviso: en este enunciado se usa como TLD para los ejemplos `.internal`. Es uno de los dominios recomendados para *Private DNS Namespaces*, entre los que podría usar `.intranet`, `.private`, `.corp`, `.home` o `.lan`. En prácticas posteriores se usará un dominio DNS real.

6. Cluster: Docker Swarm

6.1 1. Introducción

Se introduce este tema después de hacer las prácticas de Docker. Puede consultar esta referencia con una explicación de los conceptos de *stack*, *service*, *nodo*, etc: <https://docs.docker.com/engine/swarm/key-concepts/>

En la práctica se creará un cluster de 3 nodos que deben estar en la misma red: uno será el *manager* (gestor) y el resto *workers*. Para el despliegue de los tres nodos puede elegir: - Opción 1: crear tres máquinas virtuales con KVM (o VMware, Dropbox) e instalar docker en ellas. - Opción 2: crear tres instancias EC2 en AWS - Opción 3 específica de Ubuntu: usar la utilidad *multipass* para crear los tres nodos e instalar docker en ellos (ver anexo).

En esta práctica se opta por la primera de ellas.

6.2 2. Preparación del swarm

6.2.1 2.1. Nodos

- Se utiliza una máquina virtual ya preparada con Debian sin interfaz gráfico. Lleva instalada `docker`, `sudo`, `curl` y un servidor ssh.
- Como se necesitan tres máquinas virtuales lo más sencillo es preparar una y clonar las otras dos (tiene instrucciones de cómo hacerlo en forma "diferencial" `soft` o `linked` en KVM en el anexo).
- Al iniciar las máquinas se modifica el *hostname* para que sea más sencillo identificarlas.
- Se accede mediante ssh desde el *host* a las máquinas virtuales: esto refleja una situación real de trabajo (y además permite copiar y pegar).

6.2.2 2.2. Iniciar el "enjambre" o swarm

Tenga en cuenta que: * Las IPs de las máquinas virtuales dependerán de la configuración de cada sistema. * Los usuarios en las máquinas virtuales pueden tener nombres distintos: `clases`, `alu`, `usuario`, etc.

El primer paso es conectarse al nodo manager usando ssh:

```
clases@vm:~$ ssh alu@192.168.125.136
alu@192.168.125.136's password:
Linux manager 6.1.0-13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.55-1 (2023-09-29) x86_64
...
alu@manager:~$
```

El cluster se inicia con el comando `docker swarm init`. Al iniciarlo muestra el *token* de invitación para los nodos tipo *worker*:

```
alu@manager:~$ docker swarm init
Swarm initialized: current node (szbzjfi2mbaiqif01a17u0qjg) is now a manager.

To add a worker to this swarm, run the following command:
  docker swarm join --token SWMTKN-1-0fuvq18xzc68t2n584wernrf1nm0n0e3gz90cvaj4qm1a4z-81efbc8r8qxa7zhm2u1ytpv0 192.168.125.136:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
alu@manager:~$
```

Una vez iniciado puede comprobar con el comando `docker node ls` que el único nodo existente es el *manager*, que aparece como activo y con estado `Leader`

```
alu@manager:~$ docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER
szbzjfi2mbaiqif01a17u0qjg * manager        Ready     Active           Leader
alu@manager:~$
```

6.2.3 2.3. Unir nodos *worker* al "enjambre"

Para unir un nodo al cluster o enjambre es necesario usar un *token* de invitación. El *token* se muestra: * al iniciar el cluster en el manager * o ejecutando en el manager el comando `docker swarm join-token worker`.

Por lo tanto para añadir un *worker* al cluster es necesario conectarse por ssh al nuevo nodo y una vez conectado ejecutar el comando

```
docker swarm join:
```

```
alu@worker-01:~$ docker swarm join --token SWMTKN-1-0fuvq18xzc6e68t2n584wernrf1nm0n0e3gz90cvaj4qm1a4z-81efbc8r8qxa7zhm2ulytpv0 192.168.125.136:2377
This node joined a swarm as a worker.
alu@worker-01:~$
```

En el nodo *manager* puede comprobar que el nodo se ha añadido con `docker node ls`:

```
alu@manager:~$ docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER
9nhxurspobgirwh5m1d5h0fvf * manager        Ready     Active           Leader
zm7acf20ysqblhalyeicrse6w worker-01       Ready     Active
```

6.3 3. Desplegar servicio

6.3.1 3.1. Despliegue manual

Para probar el cluster **se crea en el *manager*** un servicio que llamaremos *helloworld* usando la imagen `repocurso/helloworld-node-microservice`. Esta imagen es una aplicación web que muestra el nombre (id del contenedor) del equipo.

Observe cómo se crea el servicio (en este caso se hace con una única replica) con el comando `docker service create`:

```
alu@manager:~$ docker service create --replicas 1 --name helloworld -p 8080:8080 repocurso/helloworld-node-microservice
zzcmp70yivslrzdhyxhbjblyl
overall progress: 0 out of 1 tasks
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
alu@manager:~$
```

Para probar el servicio use el comando `curl` (en vez de un navegador). Desde el *manager*:

```
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "8e088d6016c9".
alu@manager:~$
```

O desde el *worker*

```
alu@worker-01:~$ curl 127.0.0.1:8080
Hello World from host "8e088d6016c9".
alu@worker-01:~$
```

Puede acceder también desde cualquiera de los dos nodos con la IP de los nodos, y en todos los casos se accede correctamente al servicio aunque el contenedor esté sólo en uno de los nodos:

```
alu@manager:~$ curl 192.168.125.136:8080
Hello World from host "692f6380f27b".
alu@manager:~$ curl 192.168.125.141:8080
Hello World from host "692f6380f27b".
alu@manager:~$
```

Si añade otro nodo al enjambre (se conecta al nuevo nodo por ssh y ejecuta el comando `docker swarm join`):

```
alu@worker-02:~$ docker swarm join --token SWMTKN-1-0fuvq18xzc6e68t2n584wernrf1nm0n0e3gz90cvaj4qm1a4z-81efbc8r8qxa7zhm2ulytpv0 192.168.125.136:2377
This node joined a swarm as a worker.
alu@worker-02:~$
```

Puede comprobar en el *manager* cómo se ha añadido el nodo:

```
alu@manager:~$ docker node ls
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS
9nhxurspobgirwh5m1d5h0fvf * manager        Ready     Active           Leader
zm7acf20ysqblhalyeicrse6w worker-01       Ready     Active
s53vme3dp6tr2jvjmfr88avpt worker-02       Ready     Active
```

Ahora puede **escalar** a cuatro replicas el servicio usando el comando `docker service scale` e indicando el nombre del servicio y el número de réplicas:

```

alu@manager:~$ docker service scale helloworld=4
helloworld scaled to 4
overall progress: 4 out of 4 tasks
1/4: running [=====]
2/4: running [=====]
3/4: running [=====]
4/4: running [=====]
verify: Service converged
alu@manager:~$

```

Se puede observar a continuación que las replicas se distribuyen en los tres nodos, y responden a las peticiones según un algoritmo *round-robin*. Haga varias peticiones y observe cómo va cambiando el identificador del contenedor:

```

alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "eb5da9711349".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "acb9bc2b68ba".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "1f4f0b7c2822".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "8e088d6016c9".
alu@manager:~$ curl 127.0.0.1:8080
Hello World from host "eb5da9711349".
alu@manager:~$

```

Hasta ahora se han realizado las pruebas desde los nodos del cluster, pero puede acceder al servicio desde el host usando la IP de cualquiera de los nodos del cluster:

```

clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "eb5da9711349".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "1f4f0b7c2822".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "acb9bc2b68ba".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "8e088d6016c9".
clases@vm:~$ curl 192.168.125.141:8080
Hello World from host "eb5da9711349".
clases@vm:~$

```

Puede comprobar la distribución del servicio entre los nodos con el comando `docker service ps <nombreServicio>`. En este caso cada nodo "corre" un contenedor y uno de ellos dos.

```

alu@manager:~$ docker service ps helloworld

```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
z1phnld17fcz	helloworld.1	repocurso/helloworld-node-microservice:latest	manager	Running	Running 6 minutes ago		
pa5hp3zsudf3	helloworld.2	repocurso/helloworld-node-microservice:latest	worker-02	Running	Running about a minute ago		
pppd9mdjj06c	helloworld.3	repocurso/helloworld-node-microservice:latest	worker-01	Running	Running about a minute ago		
or8ythvqgb8q	helloworld.4	repocurso/helloworld-node-microservice:latest	worker-01	Running	Running about a minute ago		

```

alu@manager:~$

```

6.3.2 3.2. Borrar el servicio

Para eliminar el servicio desplegado se usa el comando `docker service rm <nombreServicio>`

```

alu@manager:~$ docker service rm helloworld
helloworld
alu@manager:~$ docker service ls

```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
alu@manager:~\$					

6.3.3 3.3. Desplegar usando stack y fichero compose

Cuando la aplicación a desplegar incluye varios servicios, volúmenes, networks, etc. es recomendable hacer el despliegue desde un fichero que tendrá un formato similar al `docker-compose` de temas anteriores. Este fichero además tendrá etiquetas específicas de un despliegue `swarm` como por ejemplo el número de replicas.

Es este ejemplo se despliega el servicio anterior usando el fichero `helloworld.yml`. Observe los campos `deploy` y `replicas`:

```

version: '3.7'

services:
  helloworld:
    image: repocurso/helloworld-node-microservice
    ports:
      - "8080:8080"

```

```
deploy:
  replicas: 2
```

Para copiar el fichero desde el host hasta el manager se usa el comando `scp fichero usuario@IP:destino`:

```
clases@vm:~$ scp helloworld.yml alu@192.168.125.136:
alu@192.168.125.136's password:
helloworld.yml          100% 160   145.2KB/s   00:00
clases@vm:~$
```

Para desplegar el servicio desde el *manager* se utiliza el comando `docker stack deploy -c <fichero.yml> <nombreStack>`

```
alu@manager:~$ docker stack deploy -c helloworld.yml demo
Creating network demo_default
Creating service demo_helloworld
alu@manager:~$ docker stack ps demo
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
v23bfy99i0yd	demo_helloworld.1	repocurso/helloworld-node-microservice:latest	worker-01	Running	Running 25 seconds ago		
vgz5h3b5gp6s	demo_helloworld.2	repocurso/helloworld-node-microservice:latest	worker-02	Running	Running 25 seconds ago		
ucsa2fut5n3v	demo_helloworld.3	repocurso/helloworld-node-microservice:latest	manager	Running	Running 25 seconds ago		

```
alu@manager:~$
```

Y para eliminar el despliegue se utiliza el comando `docker stack rm <servicio>`:

```
alu@manager:~$ docker stack rm demo
Removing service demo_helloworld
Removing network demo_default
alu@manager:~$
```

6.4 4. Herramienta gráfica: portainer

Si desea visualizar y trabajar con el cluster de forma gráfica una posible opción es usar *portainer*: <https://www.portainer.io/>. Esta aplicación despliega agentes sobre los nodos del cluster.

Para utilizarla descargamos el fichero yaml con una configuración ya preparada desde su página web con el comando `curl -L https://downloads.portainer.io/portainer-agent-stack.yml -o portainer-agent-stack.yml` y lo copiamos con `scp` al manager (o lo puede descargar directamente en el manager)

```
clases@vm:~$ curl -L https://downloads.portainer.io/portainer-agent-stack.yml -o portainer-agent-stack.yml
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed

100  791  100  791    0     0    715      0  0:00:01  0:00:01 --:--:--  726
clases@vm:~$ ls
alu@192.168.125.136 helloworld.yml portainer-agent-stack.yml
clases@vm:~$ scp portainer-agent-stack.yml alu@192.168.125.136:
alu@192.168.125.136's password:
portainer-agent-stack.yml 100% 791   1.6MB/s   00:00
clases@vm:~$
```

Y se usa el fichero desde el *manager* para hacer el despliegue:

```
alu@manager:~$ ls
helloworld.yml portainer-agent-stack.yml
alu@manager:~$ docker stack deploy -c portainer-agent-stack.yml portainer
Creating network portainer_agent_network
Creating service portainer_agent
Creating service portainer_portainer
alu@manager:~$
```

Y si observa el despliegue con `docker service ls` hay tantos agentes como nodos y un gestor:

```
alu@manager:~$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
d3ispnqym6s	portainer_agent	global	3/3	portainer/agent:2.11.1	
dynd71w6qjxn	portainer_portainer	replicated	1/1	portainer/portainer-ce:2.11.1	*:8000->8000/tcp, *:9000->9000/tcp, *:9443->9443/tcp

```
alu@manager:~$
```

Puede comprobar la distribución con `docker service ps`:

```
alu@manager:~$ docker service ps portainer_portainer
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
ad941kp4oqjg	portainer_portainer.1	portainer/portainer-ce:2.11.1	manager	Running	Running 3 minutes ago		

```
alu@manager:~$ docker service ps portainer_agent
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
d2wrk62bcgwj	portainer_agent.9nhxurspobgirwh5m1d5h0fvf	portainer/agent:2.11.1	manager	Running	Running 3 minutes ago		

```

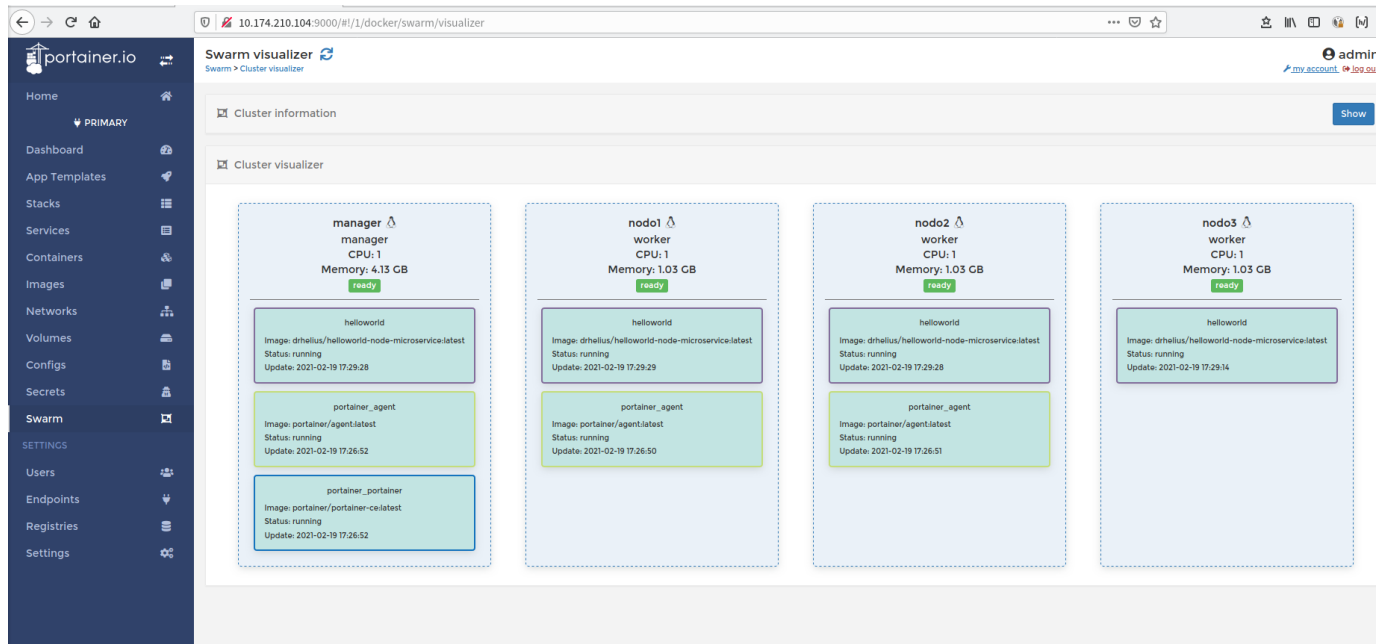
ur56o8xcj9ja    portainer_agent.s53vme3dp6tr2jvjmfr88avpt    portainer/agent:2.11.1    worker-02    Running    Running 3 minutes ago
ke16b2mwa9oy    portainer_agent.zm7acf20ysqblhalyeicrse6w    portainer/agent:2.11.1    worker-01    Running    Running 3 minutes ago
alu@manager:~$

```

Acceda ahora usando un navegador a la **IP del manager** (o cualquier otra IP del enjambre) usando el **puerto 9000** y se mostrará el tablero de gestión de portainer. En el primer acceso deberá crear un usuario y clave.

A continuación puede desde el tablero hacer despliegues de. En este ejemplo se realiza el despliegue de `helloworld` desde `portainer` usando el fichero `helloworld.yml`. Para ello acceda al "Dashboard", busque "Stacks", botón "Add Stack" y la opción "Upload from your computer".

La siguiente captura se obtiene desde "Dashboard", enlace "Cluster visualizer":



6.5 5. Anexos

6.5.1 5.1. Despliegue en AWS

Se propone como ejercicio hacer el despliegue en tres instancias EC2 de AWS.

6.5.2 5.2. KVM y clonado suave

1. Partimos de una imagen de Debian 12 con servidor ssh y sin entorno de escritorio. Lleva ya configurado `sudo` e instalado `docker`. Está disponible en la Unidad Compartida.
2. Vamos a hacer un clonado del disco de esa máquina, pero un clonado "suave" o diferencial. La idea es crear 3 máquinas que compartan como base el disco anterior:

```

[clases@d12:~/Descargas/vms]
$ qemu-img create -f qcow2 -b ./sd12-base-clone.qcow2 -o backing_fmt=qcow2 ./manager.qcow2
Formatting './manager.qcow2', fmt=qcow2 cluster_size=65536
...
[clases@d12:~/Descargas/vms]
$ qemu-img create -f qcow2 -b ./sd12-base-clone.qcow2 -o backing_fmt=qcow2 ./worker-01.qcow2
Formatting './worker-01.qcow2', fmt=qcow2 cluster_size=65536
...
[clases@d12:~/Descargas/vms]
$ qemu-img create -f qcow2 -b ./sd12-base-clone.qcow2 -o backing_fmt=qcow2 ./worker-02.qcow2
Formatting './worker-02.qcow2', fmt=qcow2 cluster_size=65536
...
[clases@d12:~/Descargas/vms]
$ ls -lh worker-* manager.qcow2 sd12-base-clone.qcow2
-rw-r--r-- 1 clases clases 193K oct 25 18:24 manager.qcow2
-rw----- 1 clases clases 3,8G oct 25 18:12 sd12-base-clone.qcow2
-rw-r--r-- 1 clases clases 193K oct 25 18:24 worker-01.qcow2
-rw-r--r-- 1 clases clases 193K oct 25 18:24 worker-02.qcow2

```

```
[clases@d12:~/Descargas/vms]
$
```

1. Creados los tres discos, creamos con `virtual manager` 3 máquinas usando los discos creados.
2. Es recomendable al iniciar las máquinas modificar el `hostname` editando los ficheros `/etc/hosts` y `/etc/hostname`.
3. Es preferible conectarse a las máquinas virtuales desde el host usando `ssh`. Anote las IPs de cada una de ellas (e incluso puede modificar el fichero `/etc/hosts` del host para asignarle nombres a las IPs de las máquinas virtuales).

Aviso: es preferible no utilizar el disco base de la clonación.

6.5.3 5.3. Uso de multipass

Aviso: Este anexo no ha sido probado este curso.

En el caso de hacerlo con la herramienta `multipass` de Ubuntu en vez de con máquinas virtuales

```
$ multipass launch -v -n nodo1 -m 512MGB # -m 1GB
$ multipass exec nodo1 -- curl -fsSL https://get.docker.com -o get-docker.sh
$ multipass exec nodo1 -- sudo sh get-docker.sh
```

Nota: evitar la instalación de docker con snap, parece que da problema al desplegar un "stack"

Para ver los nodos creados:

```
$ multipass list
Name      State   IPv4          Image
manager   Running 10.174.210.86 Ubuntu 20.04 LTS 172.17.0.1
nodo1     Running 10.174.210.224 Ubuntu 20.04 LTS 172.17.0.1
nodo2     Running 10.174.210.4   Ubuntu 20.04 LTS 172.17.0.1
$
```

Para iniciar una sesión en un nodo puede usar el comando `multipass shell`

```
$ multipass shell manager
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-65-generic x86_64)
...
ubuntu@manager:~$
```

7. Fuentes ejemplos o ejercicios

7.1 Código ejemplos de sitios web

- [2048 game](#)
- [Netflix piloto](#)

7.2 Código ejemplo tomcat

- [Código Guess Number](#)

7.3 Scripts SQL

- [Ejemplos scripts sql](#)

7.4 Docker Swarm

- [helloworld.yml](#)
- [portainer-agent-stack.yml](#)

8. Otros

8.1 Encadenar arranque contenedores

8.1.1 1. Introducción

Por defecto Docker intenta arrancar todos los servicios al mismo tiempo. Si por ejemplo una aplicación web intenta conectarse a una base de datos que todavía no está lista, la aplicación fallará.

8.1.2 2. Primer paso: `depends_on`

La instrucción para controlar el orden es `depends_on`. Con ella, se define explícitamente que un servicio (por ejemplo, web) depende de otro (por ejemplo, db). En ese caso el comando `docker compose up` arrancará los servicios en orden de dependencia: primero db, luego web. Y `docker-compose stop` los detendrá en el orden inverso.

```
services:
  web:
    image: miapp
    depends_on:
      - db # <-- Arranca 'web' solo después de iniciar 'db'
  db:
    image: postgres
    ...
```

El problema es que hay que distinguir:

- "Container Started"
- "Service Ready"

Aquí es donde muchos desarrolladores tropiezan. `depends_on` (en su forma básica) solo espera a que el contenedor esté en ejecución (running). No espera a que la aplicación dentro del contenedor esté lista (ready).

El fallo se produce si:

- Docker arranca el contenedor de Postgres (db).
- Docker ve que el contenedor está "running".
- Inmediatamente Docker arranca web.
- Postgres todavía está inicializando sus archivos y no acepta conexiones.
- la aplicación intenta conectarse, falla y se cierra (crash).

8.1.3 3. Solución: Usar `healthcheck` con `depends_on`

Esta es la forma más limpia y moderna de hacerlo si usas `docker-compose.yml`. Le dices a Docker Compose cómo comprobar la salud del contenedor de la base de datos y haces que tu aplicación espere a que esa comprobación sea exitosa.

1. Se añade un `healthcheck` al servicio de PostgreSQL. Para ello se usa la utilidad `pg_isready` que viene incluida en la imagen oficial de Postgres.
2. Usar la condición `service_healthy` en `depends_on`. En lugar de un simple `depends_on: [db]`, usar una sintaxis más avanzada.

```
services:
  db:
    image: postgres:15
    ...
    # --- INICIO DE LA SOLUCIÓN PASO 1 ---
    healthcheck:
      # Este comando comprueba si Postgres está listo
      test: ["CMD-SHELL", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"]
      interval: 5s # Comprueba cada 5 segundos
      timeout: 5s # Espera 5 segundos por una respuesta
      retries: 10 # Intenta 10 veces antes de marcar como 'unhealthy'
    # --- FIN DE LA SOLUCIÓN ---
  app:
    image: miapp
```

```

ports:
  - '8000:8000'
...
# --- INICIO DE LA SOLUCIÓN PASO 2 ---
depends_on:
  db:
    # Espera a que el healthcheck del servicio 'db' pase
    condition: service_healthy
    # El valor por defecto es service_started ( dependencia simple sin healthcheck)
# --- FIN DE LA SOLUCIÓN ---

```

8.1.4 4. Otra soluciones

Aunque la recomendada es la anterior, puede encontrar otras posibilidades.

4.1. Lógica de Reintento en la Aplicación

Esta solución implica que la aplicación sea responsable de manejar los fallos de conexión al arrancar. En lugar de intentar conectarse una sola vez y fallar si la BBDD no está lista, tu aplicación entra en un bucle:

1. Intenta conectar a PostgreSQL.
2. Si falla (ej. `ConnectionRefusedError`), espera 5 segundos.
3. Vuelve al paso 1.
4. Repite esto durante un tiempo (ej. 1 minuto) o un número de reintentos (ej. 12 intentos).
5. Si lo consigue, continúa con el arranque normal.
6. Si falla después de todos los reintentos, se cierra con un error.

4.2. Script "Wrapper" (La solución clásica, ya obsoleta.. o no)

Esta es una solución muy común que no requiere modificar el código de tu aplicación, sino solo la forma en que se inicia el contenedor.

Cómo funciona:

1. Añade un pequeño script a la imagen de la aplicación (ej. `wait-for.sh`).
2. Este script usa herramientas como `nc` (netcat) o `pg_isready` para comprobar si el puerto de la BBDD está abierto y aceptando conexiones.
3. El script se queda en un bucle hasta que la BBDD responde.
4. Una vez que la BBDD está lista, el script ejecuta el comando real de tu aplicación (ej. `npm start`, `python app.py`).

El script más famoso para esto es [wait-for-it.sh](#).

8.2 Complementos y ayudas

8.2.1 1. docker compose y variables de entorno `.env`

1.1. Usando variables del fichero `.env`

Puede utilizar variables de entorno en un fichero docker compose. Para ello debe tener en su proyecto un fichero de nombre `.env` con una lista de pares clave-valor similar a este ejemplo:

```
# fichero .env
PG_USERNAME="admin"
PG_PASSWORD="superadmin"
PG_DATABASE="shop"
```

Puede usar estas variables de entorno en su fichero docker compose usando la sintaxis `${VAR_de_ENTORNO}`:

```
# compose.yaml
services:
  db:
    image: postgres
    environment:
      POSTGRES_USER: ${PG_USERNAME}
      POSTGRES_PASSWORD: ${PG_PASSWORD}
      POSTGRES_DB: ${PG_DATABASE}
    volumes:
      - ./data:/var/lib/postgresql/data
    ...
```

1.2. Incluyendo todo el fichero: `env_file`

En el ejemplo anterior incluye las variables que necesite del fichero y se las asigna a las variables de entorno del servicio.

Otra posibilidad es pasar el fichero completo de variables a un servicio del docker compose:

Ejemplo:

```
# compose.yaml
services:
  ...
  db:
    image: mariadb
    restart: always
    env_file:
      - .env.db # Fichero con las variables de entorno que necesita el servicio
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
```

+info: * <https://www.warp.dev/terminus/docker-compose-env-file> * <https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/>